# REVON: LOG AND MERKLE DIFFERENCING FOR DURABLE STRUCTURED DATA VERSIONING

BY ATHARVA SHEERSH PANDEY[1,a], ADHYAN JAIN[1,b], AND POORNIMA NEDUNCHEZHIAN[1,c]

*1 School of Computer Science and Engineering, VIT Vellore*

*a atharva.sheersh2024@vitstudent.ac.in; b adhyan.jain2024@vitstudent.ac.in*

*c Corresponding author: Poornima Nedunchezhian (poornima.n@vit.ac.in)*

We built Revon to compare log aggregation and hash-pruned differencing over the same durable history. This Python prototype stores linear key/value histories in a SQLite-backed Merkle hash trie. We examine both diff paths, a selection rule based on operation count, and the cost of complete versioning workflows. The Linux study includes 516 executions plus a separate 72-execution supplement for geometry and telemetry. Eight of nine paired hybrid/Merkle timing intervals favour hybrid, with parity included in the dense case. In a separate threshold control, the rule chooses Merkle even though log is faster. The Windows study comprises 93 cases in fresh workers. It uses disjoint calibration and validation histories, controlled sensitivity experiments, Wine Quality and CO2 sources, and repeated runs with 100,000 and 1,000,000 records. Calibration chooses a threshold that uses log for every tested interval, leaving the value of adaptive switching unresolved. CO2 changes come from observed publications; wine corrections are generated. Semantic oracles verify diffs and historical states. We distinguish algorithm, CLI-phase and whole-worker measurements throughout. The results describe trade-offs for the tested machines and histories, without establishing an optimal crossover, storage-engine superiority or reproduction by an independent team.

**1. Introduction.** Keeping old copies of a dataset does not by itself give an application a record-level history. The application must still recover past states and compare them when only some records have changed. Git addresses file history through immutable objects identified by content and linked through parent commits [10]. Its objects represent files and directories. For a record store, keys can persist across versions even as update density and the number of intervening commits change. Saving whole files addresses part of the problem, but leaves the application to track and compare individual records.

Our comparison starts with one durable history and two ways to find its changes. The log path aggregates recorded operations by key. The Merkle path compares hashes and skips matching subtrees. They use one representation and return the same semantic diff. Revon brings together established path copying, content addressing and SQLite transactions so that we can compare these paths and a rule for choosing between them. The measurements concern the versioning operations and workloads exercised here; they do not cover the full cost of managing structured data.

The systems that motivated this comparison offer broader interfaces. Dolt combines SQL, a commit graph and Prolly Trees [13, 28]. ForkBase uses POS-Tree/SIRI storage with fork semantics [32], and Noms describes history-independent chunked collections [3]. We limit Revon to key/value operations and linear history to support a controlled

*Keywords and phrases:* data versioning, Merkle hash trie, hybrid differencing, content-addressed storage, benchmark methodology.

comparison of diff paths. We claim neither feature parity with those systems nor a new family of persistent indexes. Cross-system timings compare Revon's in-process API with Dolt's CLI. We break the CLI measurement into phases so that the included costs are visible.

*1.1. Research Questions.* RQ1: What are the latency, persistent-storage and worker-resource trade-offs in complete versioning workflows, and how does the API/CLI boundary affect how we interpret them?

RQ2: When tested on generated histories withheld from calibration, how does an operation-count selector compare with forced log and forced Merkle using the same representation?

RQ3: How do workload structure, payload size and fixed-trie geometry change differencing cost when changes to the tested histories are controlled?

*1.2. Contributions.* We implement both diff paths in a prototype that combines content addressing, path copying and atomic SQLite persistence. A shared encoded-value contract lets us compare the paths under matched conditions. The engineering contribution is the integration and the comparison it makes possible.

We also define an operation-count selection rule and test its choices against each forced path. Calibration ends, and the threshold is recorded, before we evaluate held-out histories. We retain cases where the rule makes a slower choice and examine whether using log for every request would suffice on the histories tested.

For verification, we construct oracles separately from the storage implementation and compare them with observed diffs and historical states. The accompanying materials retain executed sources, raw measurements, environment records and data-preparation steps, along with correctness checks and the documented limits of the telemetry.

The evaluation combines Linux workflow experiments with a 93-case Windows study that separates calibration seeds from validation seeds. The Windows cases cover matched sensitivity controls, Wine Quality and CO2 sources, and repeated scale runs. Each campaign has its own experimental units. We analyse its timing rows separately.

These contributions concern a single-writer store with linear histories and bounded workloads. Within that scope we can identify trade-offs. The experiments cannot establish a selector or geometry that is universally best, a superior query engine, or reproduction by an independent team.

## 2. Related Work.

*2.1. Content Addressing and Persistent Structures.* Merkle authentication ties a collection to the root that represents it [22]. Git uses content addressing for software history, and IPFS uses it for networked content [10, 5]. Path copying preserves earlier states by sharing unchanged structure instead of copying the whole collection [14]. Ordered stratified B-trees [31] and fully persistent B-trees [8] address dictionaries stored on disk. Revon instead routes hashed keys through a trie of fixed depth. Hash equality can end a comparison, but the routing does not offer ordered range access. SciDB and TimeArr use layouts and deltas suited to versioned arrays [27, 30]. TardisDB provides version access in a main-memory database, while Saquib et al. study replicated versioned structures for wide-area systems [25, 26]. These works explain relevant design choices without comparing Revon's log and Merkle paths over a common history.

Merkle Search Trees combine ordering with Merkle authentication for state reconciliation [4]. Revon shares the use of hashes to prune comparisons, but its routing has

fixed depth and its history is centralized and linear, rather than a replicated CRDT. Another approach to sharing appears in LBFS, which finds redundant content through content-defined chunking [23]. Dolt and Noms also use related methods to determine chunk boundaries. Those boundaries can retain deduplication when data shifts; ordered indexes support range operations. Our benchmark evaluates neither capability.

*2.2. Structured Dataset Versioning.* Storing and recovering versions is only part of dataset versioning: collaborative work matters too. DataHub studies collaborative histories at scale [6]. Decibel provides relational branching and compares version-first, tuple-first and hybrid layouts [20]. OrpheusDB improves version retrieval through partitioning when versioning is added to an existing DBMS [19]. Bhattacherjee et al. formalize the balance between storage and reconstruction [7]. Delta Lake builds ACID tables on transaction logs and checkpoints [2], while version-graph methods choose materialized states and deltas within storage and retrieval constraints [18]. The FAIR-compliant proposal concerns consistent identification and description of releases [16]. It complements storage work without prescribing a diff algorithm. Revon's key/value interface covers a narrower part of this dataset-management problem.

*2.3. System Design Gap.* In Noms, a Merkle DAG represents structured values and supports diff and synchronization [3]. ForkBase adds fork and deduplication semantics to content addressing [32]. Dolt's content-addressed Prolly Trees provide structural sharing, native diff and ordered indexing [13, 28]. Git4Data exposes snapshot, branch, diff and merge through SQL over a relational engine [17]. We cite its available arXiv preprint for the abstraction it describes, without treating the reported results as peer-reviewed or independently reproduced. Revon's implementation is deliberately narrower: a single-writer key/value store, linear history and fixed-depth hash routing, with no SQL or branching interface.

Table 1 lists the systems and studies closest to the implementation choices made here.

*Table 1*
*Summary of related work*

| System or study | Representation or focus | Relevance to Revon |
|---|---|---|
| Persistent DS [14] | Path copying | Formal structural-sharing basis |
| Git [10] | Object DAG | Identity and commit ancestry; file-oriented |
| IPFS [5] | Merkle DAG | General content-addressed versioning |
| MST [4] | Merkle search tree | Authenticated ordered reconciliation |
| DataHub [6] | Version graph | Collaborative dataset motivation |
| Decibel [20] | Relational layouts | Native database branching |
| OrpheusDB [19] | Partitioned versions | Storage/retrieval optimization |
| Delta Lake [2] | Log/checkpoints | ACID time travel at data-lake scale |
| Noms [3] | Merkle DAG | Structured content addressing |
| ForkBase [32] | Forkable CAS | Fork and deduplication semantics |
| DoltHub [13, 28] | Prolly Trees | Mature production comparator |

*2.4. Design Implications and Research Positioning.* A representation determines where the work falls as versions are created, retained, reconstructed and compared. With complete snapshots, retrieval is simple but unchanged records are copied. With logs, some work is deferred to replay or checkpointing. Path copying reuses unchanged structure and rebuilds the paths touched by an update [14, 7]. Decibel and OrpheusDB examine the effect of layout on version creation and retrieval [20, 19]; version-graph analyses express the storage/retrieval trade-off directly [18]. Work on reproducibility asks how builds and artifacts can remain available for inspection [1]. Accordingly, we measure import, commit, checkout, diff and storage separately. Preserving the artifacts supports inspection, not a claim of faster execution.

The output of a diff also needs to be defined. Differential dataflow updates query results as timestamped inputs change [21]. Provenance asks where output facts originate and why they occur [9]. DSLog records fine-grained array lineage and supports queries over it [33]. Explain-Da-V connects semantic dataset changes to transformations [29]. Baseline's operation-based differencing handles refactorings and schema changes [15]. Our output is simpler: keys added, removed or modified between two states. Revon does not infer causes, lineage, query effects or schema intent. The related work supplies context, but its different outputs and workloads cannot support direct timing comparisons with these key/value diffs.

Revon-H retains a Merkle root and a content-addressed changeset at every commit. Either path can recover the endpoint changes, so selection affects the method used to compute the diff. Our hypothesis is that log aggregation will suit short, sparse intervals and that hash pruning may help after longer operation sequences. The measurements must determine whether the costs cross. Git4Data reflects a broader interest in database-native version control [17]. We use Snapshot and Log-only as representation baselines for the fixed-depth trie. The public workloads apply generated corrections to TLC and wine records, and use observed changes in a CO2 publication sequence. They do not establish performance on independent hardware, under concurrency or in production.

*2.5. Capabilities and comparison boundaries.* Table 2 separates descriptions of system abstractions from the evidence collected for this paper. Its capabilities are documented features, not benchmark results. ForkBase and Noms describe typed-collection APIs; we evaluate no relational layer over them. Git4Data appears through a preprint at a different abstraction level [17]. Leaving a capability unevaluated does not mean that the system lacks it.

*Table 2*
*Documented system capabilities and the scope of the evidence; features do not establish performance*

| System | Representation / history | Diff / persistence | Branches / queries | Reproducibility and use here |
|---|---|---|---|---|
| Revon | Fixed-depth hash trie; linear history | Log or Merkle; SQLite objects | No named branches/merge; key/value API | Executed snapshots, semantic proofs and raw trials; measured here |
| Dolt [13, 28] | Prolly Trees; commit graph | Native version diff; persistent block store | Branches/merge; SQL | Official docs/source; CLI workflow measured, engine not isolated |
| ForkBase [32] | POS-Tree/SIRI; forkable versions | Hash-pruned diff/merge; addressed chunks | Fork on demand/conflict; typed KV APIs | Primary paper and APIs; no experiment run here |
| Noms [3] | History-independent chunks; Prolly collections | Hash-based diff/sync/merge; chunk store | Typed collection API; not benchmarked here | Technical overview/source; conceptual comparison |
| Git4Data [17] | MatrixOne immutable objects/MVCC; table versions | Database-native diff/merge | Snapshot/tag/branch ; SQL extensions | Preprint/BranchBench claims belong to its authors; not run here |

### 2.6. Related foundations and scope boundaries.

Version identity and descriptions of change are part of reliable citation and reuse in research-data versioning guidance [34]. The IPFS evaluation and FastVer study content-addressed persistence and integrity in settings broader than our local prototype [35, 36]. Persistent tries, B+-trees, external-memory search trees, snapshot stores and authenticated data structures retain and verify earlier states in different ways [37-41]. Studies of repository views, collaborative analytics and DEX show how history systems serve repository access and collaborative work [42-44].

Changes to data meaning and structure are studied in schema evolution and temporal databases [45-47]. Provenance research follows the origins and derivations of results through semirings, scientific-workflow surveys, tutorials and wider syntheses [48-51]. Other work covers aggregate-query provenance, authenticated or secure provenance, FAIR data stewardship, privacy and the PROV model [52-56]. SEEK provides an example of provenance integrated with data management in a scientific repository [57].

The tree and log mechanisms used here belong to a wider storage literature. B-tree foundations and cache-oblivious indexing examine alternatives in search and update cost [58-61]. LSM trees, Bigtable, Dynamo and log-structured file systems organize storage around logs or appends [62-65]. WiscKey and LogBase examine write-efficient key-value and log-based database storage; multi-version partitioned B-trees address indexing across versions [66-68]. These systems motivate comparisons between designs. Their measurements and workloads, however, cannot stand in for the ones evaluated in this paper.

## 3. Revon Design.

Figure 1 locates the evaluated operations within Revon's implemented architecture.

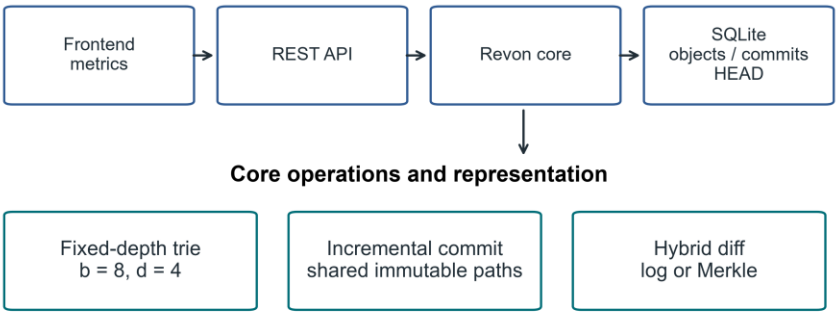



*Fig 1. Implemented Revon architecture and the scope of the research prototype*

*3.1. Addressed Object Model.* A trie node or changeset is serialized as canonical JSON and identified by the SHA-256 digest of that object. A commit's digest covers its root, parent, changeset, message and timestamp. Labels such as v1 and v2 are human-readable aliases; changing a label does not change object identity. Revon records the current commit through a durable HEAD reference.

*3.2. Fixed-Depth Merkle Hash Trie.* The main configuration sets b=8 and d=4. Routing takes three bits from SHA-256(key) at each level, using twelve bits to select among 4,096 possible leaf buckets. A leaf keeps the full key, so a shared routing prefix does not make two keys indistinguishable. Uniform hashing gives expected occupancy N/b^d. We test other combinations of b,d separately, without presuming that this default has the lowest cost.

*3.3. Incremental Hashing and Commit.* Revon first canonicalizes a batch's values and groups keys by their routing prefix. It rewrites each affected leaf once and rebuilds ancestors along the touched paths. Where a child hash is unchanged, the internal node retains it and shares the corresponding structure. One SQLite transaction writes the objects, changeset, commit metadata, version alias and measurements, then updates HEAD. When the repository is reopened, stored payload hashes and references are verified.

*3.4. Hybrid Diff.* The log path walks the requested ancestor interval and composes operations for each key. The Merkle path compares roots recursively: equal hashes stop traversal, while unequal paths lead to entry comparisons. Before choosing a path, Revon-H validates version identifiers, checks ancestry in both directions and counts changeset operations. An eligible ancestor interval goes through log when its count is no greater than T; otherwise it goes through Merkle. Reverse log requests exchange old and new values. Comparing a version with itself gives an empty diff. Forced log rejects unrelated versions, and unknown identifiers raise an error. Revon-M always requests Merkle, although ancestry is still checked by its public version-pair API. Both paths return the same sorted semantic diff. Commit and checkout have separate timers.

For the Linux workflow matrix, the protocol fixes T=4,096. An auxiliary calibration experiment supplies T=16,384 for a separate Linux control. These thresholds count operations. Bucket capacity comes from b^d, a separate parameter. Neither is fitted to the Linux outcomes.

*3.5. JSON value ownership and identity.* The accepted values are JSON nulls, booleans, finite numbers, strings, lists and objects with string keys. Revon validates input containers and detaches them before storing them. It also detaches containers returned by reads or diagnostic views, although immutable scalars can be shared. Unsupported Python values, nested non-string keys, non-finite numbers and cyclic containers are rejected before writing. Equality follows the v1 JSON encoding: object keys are sorted, separators are compact, text uses UTF-8 and NaN is disallowed. As a result, true, 1, 1.0 and signed floating zero keep distinct encodings. This defines the prototype's Python encoding contract; it does not assert compliance with a cross-language canonical-JSON standard.

Algorithm 1 (batch update). Validate the batch and take ownership of its values. Before interning any object, reject a key listed in both puts and deletes, or a request to delete a missing key. Look up old values and retain only mutations that change encoded identity. Group the retained mutations by routing prefix. Rewrite each touched leaf, sort its entries and hash the complete payload. Rebuild the affected ancestors, keeping unchanged child hashes wherever possible, then record the content-addressed changeset and commit. Because values are detached, callers cannot change stored history through their input containers or through supported reads.

Algorithm 2 (log composition). Traverse the ancestor interval chronologically. For each key, remember whether it exists and its value at the first old endpoint and at the last new endpoint. Remove the key from the result only if both existence and encoded value match at those endpoints. Thus an insertion followed by deletion cancels, as does an update followed by restoration. A null value still differs from a missing key. Sort the remaining keys. If the request runs in reverse, exchange the endpoints and the added/deleted labels.

Algorithm 3 (selection). Validate the versions and determine whether either is an ancestor of the other. Sum the changeset operation counts across the interval. An eligible interval uses log when the sum is at most T; all other cases use Merkle. During Merkle traversal, skip equal-hash subtrees and compare values in unequal leaves. Use the same encoded-identity rule for both paths and detach values before returning them.

Cost model. We write N for the record count, d for depth, b for the branching factor, m for keys in a batch, h for interval length, p for accumulated operations, u for distinct interval keys and s for serialized output bytes. Under uniform routing, a leaf has expected occupancy N/(b^d); its worst-case occupancy is N. A batch requires validation and copying of inputs, m routed lookups of old values and scans of occupied leaves. It must also sort and serialize touched leaves and copy affected internal child maps. Those costs rule out a dataset-independent O(d) bound for batch processing.

Composing a log diff takes O(p + u log u) work while walking ancestry, with encoded-value comparisons and output copying/serialization adding to that cost. Merkle work depends on how many node pairs are visited, how child slots are sorted and which leaf entries are examined. Comparing values and sorting output add further work there too. Ancestry traversal occurs in both public version-pair modes. For reverse requests, the direction that fails may first walk back to genesis. Building the output takes at least work proportional to s. Benchmarks limited to scalar values do not capture the extra comparison and copying costs of large JSON values.

The retained history has to fit in memory. Revon eagerly loads nodes, changesets and commits from SQLite, so objects remain resident during execution even though they are durable on disk. Retained payload also affects the work of opening and verifying a repository. There is no disk-page cache or eviction policy. SQLite transaction atomicity is the persistence mechanism used here. Concurrency and operational guarantees for production would need evaluation beyond these tests.

Figure 2 follows a version-pair request through eligibility checks, including the reverse-request case, to path selection.

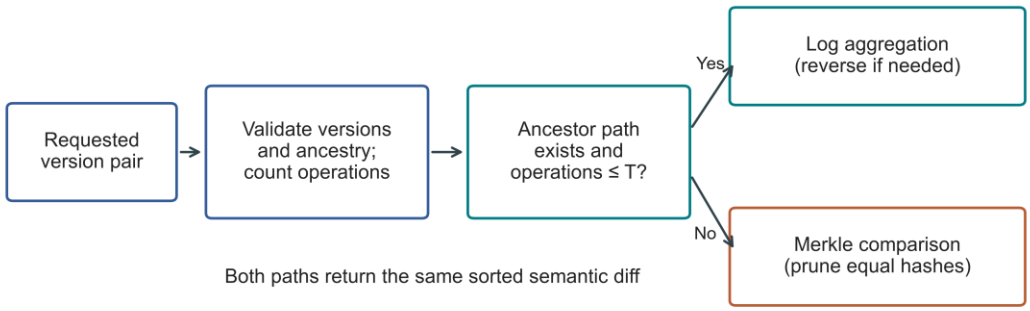



*Fig 2. Selection of a diff path for a version pair. T is the operation threshold. Invalid versions raise an error before selection; both paths share one persistent trie*

## 4. Experimental Methodology.

*4.1. Environment and evidence boundaries.* We used an Ubuntu n2-standard-4 cloud VM for Linux and a separate author-operated computer for Windows. Table 3 gives the Linux environment and identifies the executed sources. Hardware, OS, runtime and storage differ together across the campaigns, so we cannot attribute a difference between them to a single factor. The authors ran both campaigns.

*Table 3*
*Linux environment and records of the executed sources*

| Property | Recorded configuration |
|---|---|
| Machine | GCP n2-standard-4; Intel Cascade Lake; 4 vCPUs; 16 GB RAM; us-east1-c |
| Operating system | Ubuntu 24.04.5 LTS; x86_64; kernel 7.0.0-1011-gcp |
| Storage | 100 GB balanced persistent disk; no swap configured |
| Runtime | CPython 3.14.3, Clang 22.1.1 build; GIL enabled; JIT not enabled; SQLite 3.50.4 |
| Comparator and packages | Dolt 2.3.1; psutil 7.1.0; PyArrow 25.0.1 |
| Primary source | linux-validation-20261008: executed source, fixed before measurement |
| Supplement source | linux-final-supplement-20261008: same core; zero-aware I/O sampler and geometry history checks |
| Background conditions | Google Ops Agent active; filesystem cache not cleared; monitoring included |

For every campaign we retain hashes of the executed sources, the fixed protocol, environment, correctness checks and raw observations. The Linux preflight suites passed 67 and 69 tests, and the Windows suite passed 77. The records retain setup failures and the public-data exclusion applied before timing. Access instructions and detailed procedures are given in Appendices A and B.

*4.2. Variants and workloads.* Five systems produce six measured workflows. Snapshot writes a complete state for each version. Log-only keeps an initial state followed by operations. Revon-M requests Merkle diff and Revon-H selects a path. Dolt stores the records in a primary-key SQL table and returns its native JSON diff. We treat SQL INSERT and CSV table import as separate Dolt workflows. Despite their different interfaces and storage features, subsequent operations follow one semantic-output contract. Table 4 defines the workload IDs appearing in the figures.

*Table 4*
*Linux workloads; every primary history contains ten generated commits*

| ID / scenario | Rows | Changes / commit | Locality |
|---|---|---|---|
| S1 Small sparse | 1,000 | 10 | Spread |
| S2 Medium sparse | 10,000 | 10 | Spread |
| S3 Medium dense | 10,000 | 1,000 | Spread |
| S4 Large sparse | 100,000 | 100 | Spread |
| S5 Application-key local | 100,000 | 100 | Lexically adjacent keys |
| S6 Range local | 100,000 | 100 | Contiguous key intervals |
| S7 Repeated key | 100,000 | 100 | Repeated small key set |
| S8 Hash-route local | 100,000 | 100 | Shared 9-bit hash prefix |
| P1 TLC public records | 100,000 | 100 | Generated fare corrections |

The execution count comes from eight synthetic scenarios, each with six variants, two warm-ups and seven measured trials: 432 executions. Public-record workloads add 27 across Revon-M, Revon-H and Dolt SQL, and threshold controls add 54. Three million-row diagnostics complete the total of 516, split into 114 warm-ups, 399 measured trials and three diagnostics. Every execution completed with a verified output. The accounting neither silently substitutes for failed timing executions nor omits them.

*4.3. Public dataset and generated histories.* We take TLC data from the January 2024 yellow-taxi release [24], fixing the 2,964,624-row Parquet input by checksum. With seed 20261001, a dispersed affine permutation gives nested subsets of 10,000, 100,000 and 1,000,000 rows. We retain eleven fields: vendor, pickup/drop-off times, passenger count, distance, pickup/drop-off locations, payment type, fare, tip and total. A key combines the release, file-checksum prefix and physical row ordinal. This identifies a row inside the file, without asserting that the same trip can be identified across releases. Values use compact UTF-8 JSON with sorted keys, including nulls. The million-row subset has 47,277 missing passenger counts, yet every row serializes successfully. Rows are not merged when field values happen to match. Sampling by this affine procedure is not uniformly random.

Each generated TLC history has ten commits. A Linux commit changes 100 records in the 100,000-row workload, or 1,000 records in the million-row diagnostic. We add 0.01 to fare and total for each correction and keep all other retained fields unchanged. Every system receives the same inputs and mutations. These are generated edits, not observed

revisions of trips. The public-data builder retains all eleven source fields and ignores the inherited payload_bytes=8 configuration field; payload size comes from measured serialized bytes. We store a canonical record as a JSON-encoded string in the common value column. Consequently, the experiment covers neither native nested-object mutation nor queries over individual fields.

*4.4. Timing, correctness and uncertainty.* A workflow starts with preparation and repository construction, then proceeds through timed operations, correctness verification and analysis. Import, commit, diff and checkout are timed separately. The common contract requires diff to emit sorted keys and old/new-value hashes. Table 5 distinguishes algorithm work from interface costs and measurements of the whole worker. We run correctness checks and decision probes outside the operation timers.

*Table 5*
*Measurement boundaries and their interpretation*

| Quantity | Revon / same-representation control | Dolt CLI workflow | Interpretation |
|---|---|---|---|
| Diff latency | Version API, selected path and canonical output | Native JSON-diff command, parsing and canonical output | Complete interface contract; no engine-only ranking |
| Phase probe | Separate ancestry/count probe; includes detached-view copying | Command, JSON parse and output-construction timers | Neither probe nor command isolates pure tree-engine work |
| Correctness | All states, reverse/identity, integrity and reopen outside timers | Observed native rows and historical checkout outside timers | Observed values compared with an independently constructed oracle |
| Resources | Worker plus sampled children, including oracle/setup | Worker plus sampled Dolt processes | 10 ms samples; short processes/peaks may be missed |
| Persistent bytes | Windows: closed files; Linux: before close/compaction | Repository directory files | Includes metadata/indexes; not isolated algorithm overhead |

Observed endpoint diffs and every historical state are checked against workload oracles constructed apart from the storage implementation. For Revon we additionally check reverse and identity requests, stored-object integrity and reopening. The audits reconstruct the plan and semantic hashes to check that the records are internally consistent. They do not repeat the timed experiment.

The Linux unit is a fresh-repository trial on a fixed history: we use seven trials and paired percentile-bootstrap timing intervals. Within a Windows case, seven cached queries share a fresh repository. We report medians across generated histories separately from medians across repository repetitions. These units are different, and neither campaign samples a population of machines.

The sampler observes the worker and its descendants during setup and verification as well as storage operations. Its CPU and memory totals therefore extend beyond the storage implementation. Primary Linux read-counter records cannot distinguish supported zero readings from missing counters. The Linux supplement and Windows sampler make that distinction explicitly. Even so, Windows process I/O bytes do not measure direct physical-disk traffic.

Persistent metadata and representation costs are included in repository footprint. For Linux we measure directory size before closing or compacting; for Windows we measure

after closing and the specified verification steps. We keep the results separate because those conditions differ. Neither result is tree overhead in isolation.

*4.5. Frozen threshold and geometry controls.* Auxiliary calibration on Windows chose T=16,384, the largest candidate it tested. That choice did not identify a crossover. We carry it unchanged into the Linux controls. The Windows selector experiment described below uses a separate fixed design and executed source, with disjoint calibration/validation seeds, so we analyse its threshold findings separately.

The Linux control fixes 10,000 rows and compares forced log, forced Merkle and hybrid at T=16,384. Its intervals contain 8,192 and 32,768 accumulated operations. We neither choose nor retune this control from Windows outcomes.

For Linux geometry, we test five configurations against one fixed spread workload. Windows sensitivity testing instead uses three generated histories and matched geometries of equal capacity. Bucket capacity remains separate from the operation threshold. These experiments describe the configurations tested without establishing a generally optimal geometry.

*4.6. Windows calibration and validation.* We fix N=12,000, H=8, payload=32 bytes and b=8,d=4 for the Windows selector comparison. Spread/repeated-key histories have 64, 512 or 4,096 changes per commit. Two seeds are used for calibration and three different seeds for validation. Within each case, forced log, forced Merkle and hybrid thresholds T=512/4,096/16,384/65,536 share the same durable representation. We deterministically shuffle configuration order, run two warm-ups and then measure seven query blocks. Cases start in fresh workers. Queries inside a case reuse resident objects and the filesystem cache, so the experiment does not measure a cold cache.

To choose T, we divide the median of each hybrid case by the lower forced-path median and minimize the mean of those ratios, weighting cases equally. If candidates tie exactly, the smaller T wins. We record the selected threshold and UTC selection time before the first validation worker starts. Validation does not choose a new winner. Prespecified sensitivity stages vary rows, history interval, payload, geometry and routing. Holding accumulated operations constant couples interval length to changes per commit, so this is not a fully factorial design. Seeds and matched controls are listed in the appendices and protocols.

*4.7. Public histories and repeated scale experiments.* For Wine Quality, keys combine colour with source row ordinal across 6,497 red/white records [11]. We generate three histories, each containing eight batches of 64 seeded quality-field corrections. These edits are not observed specimen revisions. For CO2 we use published NOAA-derived CSV revisions from the package [12]. Before fixing the experiment, we acquire the eleven most recent snapshots in which the file changes and exclude one publication failure that contains only a header. The remaining ten populated snapshots give nine observed transitions from an initial 812 records. Unnamed trailing columns are kept as extra fields. Records are identified by date; CSV order and formatting are excluded from state comparison. The history records repository publications, not NOAA transactions or evidence about atmospheric science.

All three CO2 repositories replay that one observed sequence. The wine and scale repositories use different generated seeds. For each public source we measure Revon-M, T=4,096 Revon-H and native Dolt JSON diff, rotating model order across runs. The scale cases set H=4, C=64 and payload=32 for 100,000 and 1,000,000 records, with three seeds for each size/system. We describe their repository medians through medians and full

ranges. Source URLs, licences, acquisition versions, checksums and transformations are recorded in the metadata. Acquiring a bounded publication history was feasible on the available host without collecting Software Heritage in bulk.

Appendix B gives the execution-count matrix in Table 17 and the TLC checksums and Linux public-data protocol in Table 16. Timing boundaries and experimental units are defined in the main text, allowing the results to be read without rebuilding the complete execution manifest.

## 5. Linux Results.

*5.1. Hybrid versus forced Merkle.* This ablation holds the stored representation constant. Revon-M and Revon-H retain identical persistent structures and changesets; only the requested diff strategy differs. Figure 3 and Table 6 contain all nine comparisons with repeated trials. Eight hybrid intervals exclude parity. For dense S3, the interval includes one. The sparse and repeated-key histories benefit from log access in diff execution, but choosing log does not make the stored representation smaller. The comparison does not show that switching helps every workload.

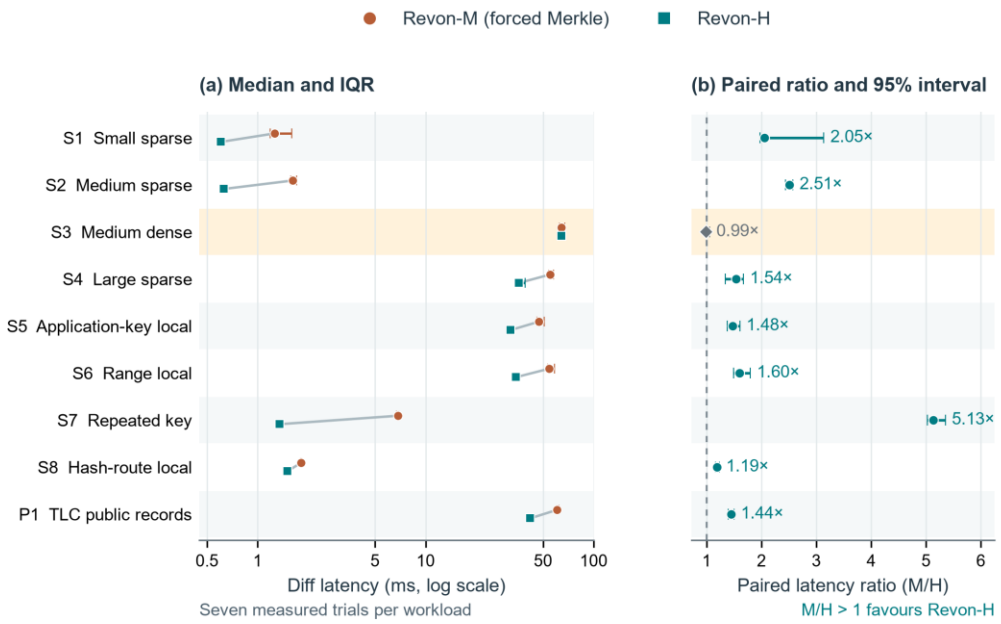



*Fig 3. Revon-M and Revon-H diff timings on Linux. Panel (a) connects workload pairs and shows median latency with interquartile ranges on a logarithmic axis. Panel (b) shows median matched-trial M/H ratios with paired percentile-bootstrap 95% intervals from 20,000 resamples. Parity is the dashed line. Each workload has seven measured trials. The highlighted S3 interval includes parity, while the other eight favour H. A median of trial ratios can differ from a ratio of latency medians. Table 4 defines the workloads. The intervals concern repeated timings of fixed histories on one VM, rather than variation across independent datasets or machines.*

*Table 6*
*Paired M/H diff comparison: milliseconds and median trial ratio [95% interval]*

| ID | M (ms) | H (ms) | M/H ratio [interval] | H path |
|---|---|---|---|---|
| S1 | 1.259 | 0.604 | 2.05 [1.97, 3.14] | log |
| S2 | 1.620 | 0.629 | 2.51 [2.44, 2.57] | log |
| S3 | 64.135 | 64.183 | 0.99 [0.97, 1.06] | merkle |
| S4 | 54.897 | 35.749 | 1.54 [1.34, 1.67] | log |
| S5 | 47.253 | 31.967 | 1.48 [1.38, 1.60] | log |
| S6 | 54.352 | 34.537 | 1.60 [1.50, 1.80] | log |
| S7 | 6.862 | 1.345 | 5.13 [5.02, 5.35] | log |
| S8 | 1.817 | 1.503 | 1.19 [1.17, 1.23] | log |
| P1 | 60.597 | 41.964 | 1.44 [1.39, 1.50] | log |

For P1, the median diff takes 41.964 ms with Revon-H and 60.597 ms with forced Merkle. The inputs are the same eleven-field public records with the same generated edits. H selects Merkle on S3 because the interval accumulates 10,000 operations. The near-parity finding fits a common diff path with selection work added in H.

*5.2. Threshold control and adverse result.* A slower choice is visible in Figure 4 and Table 7. Hybrid takes log at 8,192 operations, but switches to Merkle at 32,768. At that larger count, forced log wins all seven matched measured trials. These controls therefore show a case where operation count fails to choose the fastest path. They test the fixed threshold on Linux and leave the best threshold unresolved.

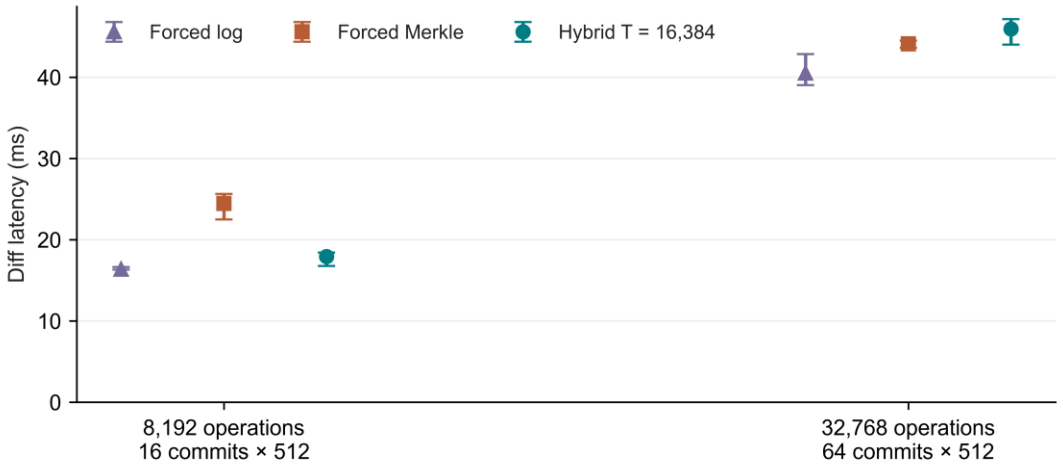



*Fig 4. Linux controls with T=16,384 fixed. Each path has seven measured trials; points show medians and bars show IQRs. Labels distinguish the history lengths*

*Table 7*
*Fixed-threshold controls with diff medians in milliseconds*

| Operations / history | Forced log | Forced Merkle | Hybrid | Selected path |
|---|---|---|---|---|
| 8,192 / H=16 | 16.364 | 24.466 | 17.926 | log |
| 32,768 / H=64 | 40.520 | 44.120 | 45.944 | merkle |

*5.3. Complete workflow comparison.* Table 8 measures the full workflow that produces a canonical diff through Revon-H's API or Dolt's SQL/CLI interface. All nine paired Dolt/H intervals exceed one. These timers include data structures, process startup, language, serialization, query execution and resident state. They cannot separate storage-engine performance from those costs. Commit and checkout are shown separately in Figure 5 because costs differ by operation.

*Table 8*
*Dolt SQL/CLI versus Revon-H diff: milliseconds and paired ratio [95% interval]*

| ID | Dolt (ms) | H (ms) | Dolt/H ratio [interval] |
|---|---|---|---|
| S1 | 90.704 | 0.604 | 148.15 [139.77, 160.15] |
| S2 | 95.515 | 0.629 | 148.70 [140.08, 150.80] |
| S3 | 297.617 | 64.183 | 4.70 [4.54, 4.77] |
| S4 | 152.472 | 35.749 | 4.25 [3.84, 4.34] |
| S5 | 125.594 | 31.967 | 3.73 [3.71, 3.87] |
| S6 | 135.765 | 34.537 | 3.88 [3.69, 4.12] |
| S7 | 97.451 | 1.345 | 71.73 [68.95, 74.32] |
| S8 | 110.966 | 1.503 | 71.24 [68.83, 77.06] |
| P1 | 168.576 | 41.964 | 4.02 [3.77, 4.07] |

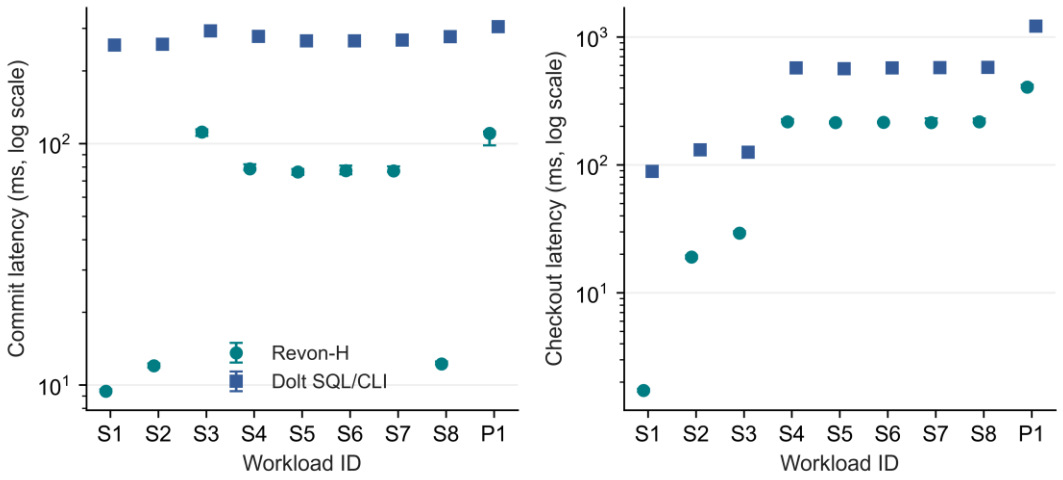



*Fig 5. Incremental-commit and full-checkout latency on Linux for Revon-H and Dolt SQL/CLI. Points show medians and bars show IQRs from seven trials. Both latency axes are logarithmic*

*5.4. Storage and memory costs.* Latency is only one part of the measured cost. Figure 6 and Table 9 also give persistent directory size and peak RSS for the entire worker. Both M and H retain content-addressed changesets and trie data. Their small difference in directory size arises from the measured workflows, not from deleting data when log is chosen. Dolt differs in serialization, compression and indexing as well. A faster diff may thus accompany a much larger repository; storage and worker resources belong in the efficiency comparison.

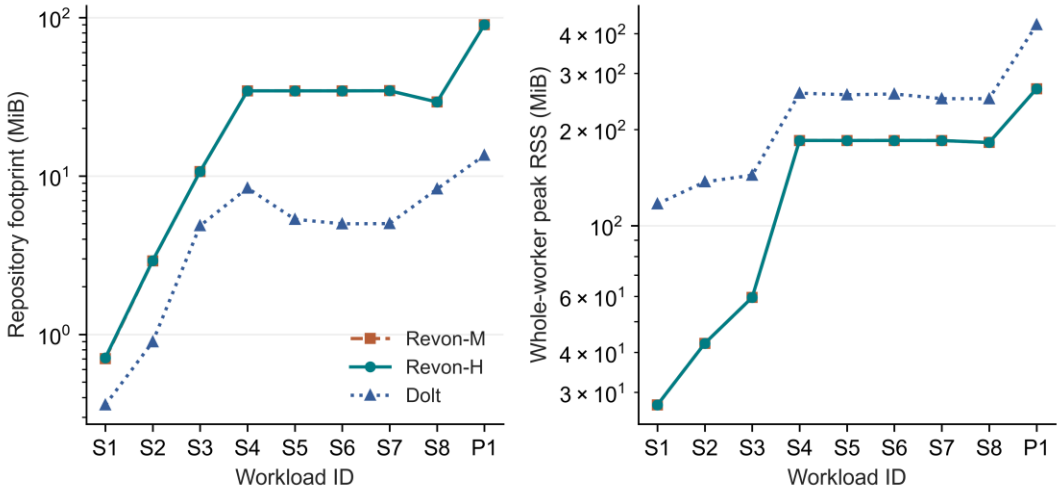



*Fig 6. Linux medians over seven trials: repository size before compaction and whole-worker peak RSS, both on logarithmic axes. RSS includes shared workload and verification memory*

*Table 9*
*Repository footprint and worker RSS, reported as medians in MiB*

| ID | M disk | H disk | Dolt disk | H RSS | Dolt RSS |
|---|---|---|---|---|---|
| S1 | 0.7 | 0.7 | 0.4 | 27.4 | 116.9 |
| S2 | 2.9 | 2.9 | 0.9 | 42.7 | 137.1 |
| S3 | 10.7 | 10.7 | 4.8 | 59.5 | 143.9 |
| S4 | 34.4 | 34.4 | 8.4 | 184.9 | 260.1 |
| S5 | 34.4 | 34.4 | 5.3 | 184.8 | 257.2 |
| S6 | 34.4 | 34.4 | 5.0 | 184.9 | 258.6 |
| S7 | 34.5 | 34.5 | 5.0 | 184.8 | 250.0 |
| S8 | 29.3 | 29.3 | 8.3 | 182.2 | 249.9 |
| P1 | 90.1 | 90.0 | 13.5 | 268.6 | 426.5 |

*5.5. Million-record feasibility.* We construct one fresh repository each for M, H and Dolt in the Linux million-row diagnostic, verifying all eleven historical states and the common diff oracle. Table 10 reports these individual observations without IQRs, confidence intervals or a ranking. With 10,000 accumulated operations above T=4,096, hybrid uses Merkle. The runs establish that the bounded workload completes on the VM. One observation per system cannot establish relative scaling or an optimal configuration. The separate repeated Windows scale study appears in Section 6.4.

*Table 10*
*Million-record diagnostics with n=1 per system; individual observations without comparative inference*

| System | Import (s) | Commit (ms) | Diff (ms) | Checkout (s) | RSS (GiB) |
|---|---|---|---|---|---|
| Dolt | 24.902 | 547.311 | 817.034 | 11.454 | 2.555 |
| Revon-H | 32.289 | 2961.901 | 1233.601 | 4.729 | 2.561 |
| Revon-M | 32.454 | 2979.717 | 1265.367 | 4.685 | 2.561 |

*5.6. Fixed-trie geometry and telemetry.* Figure 7 and Table 11 compare geometry configurations on Linux. Each of b4-d6, b8-d4 and b16-d3 gives 4,096 leaf buckets. Their equal expected occupancy lets us examine routing shape at a fixed capacity. When b=8 is held constant, depth changes capacity as well as the balance between traversal and leaf scanning. This experiment holds one workload and one VM fixed, so it cannot establish a geometry that is generally optimal.

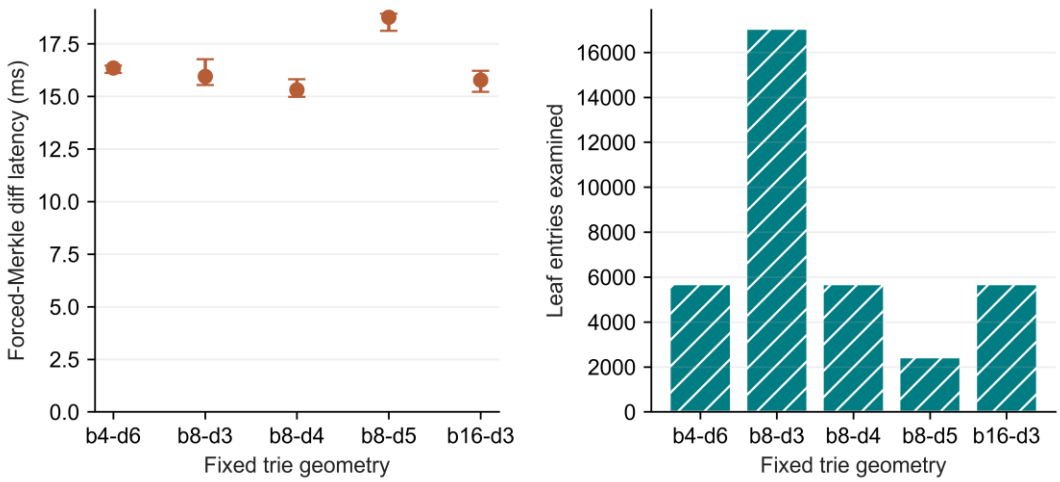



*Fig 7. Linux fixed-trie geometry. The left panel gives forced-Merkle diff medians and IQRs from seven trials. The right gives the median count of examined leaf entries*

*Table 11*
*Geometry work and footprint for one fixed spread workload*

| Geometry | Buckets | Diff (ms) | Leaf entries | Node pairs | Disk (MiB) |
|---|---|---|---|---|---|
| b4-d6 | 4096 | 16.355 | 5682 | 3628 | 4.32 |
| b8-d3 | 512 | 15.949 | 17048 | 585 | 6.88 |
| b8-d4 | 4096 | 15.316 | 5682 | 3729 | 4.25 |
| b8-d5 | 32768 | 18.765 | 2426 | 6089 | 6.18 |
| b16-d3 | 4096 | 15.775 | 5682 | 3929 | 8.12 |

Both isolated-worker telemetry tests pass, distinguishing valid all-zero I/O samples from counters that are unavailable. The Linux telemetry supplement has 27 executions on a fixed 10,000-row workload. M, H and Dolt each run two warm-ups and seven measured trials. A measured zero read-byte value is retained explicitly instead of becoming a blank. This establishes the handling of counters, not cold-storage behaviour or freedom from missed samples. We analyse the primary latency summaries separately.

*5.7. Representation baselines.* We use the spread workload S4 to illustrate all six workflows through descriptive medians in Table 12. Appendix A points to the GitHub reproducibility archive containing the complete eight-scenario matrix, trial dispersion, normalized storage and separate SQL/CSV import workflows. In the common harness, Snapshot and Log-only control for representation without matching every production feature. Their costs for complete-state storage and operation replay provide context for Revon.

*Table 12*
*Workflow medians on representative S4; the complete eight-scenario matrix is in the GitHub reproducibility archive*

| Workflow | Import (s) | Commit (ms) | Diff (ms) | Checkout (ms) | Disk (MiB) |
|---|---|---|---|---|---|
| Snapshot | 0.071 | 69.577 | 624.860 | 168.158 | 53.5 |
| Log-only | 0.072 | 2.474 | 16.483 | 159.538 | 5.0 |
| Revon-M | 1.488 | 78.842 | 54.897 | 215.661 | 34.4 |
| Revon-H | 1.644 | 78.590 | 35.749 | 217.482 | 34.4 |
| Dolt SQL | 1.663 | 277.593 | 152.472 | 572.585 | 8.4 |
| Dolt CSV | 1.335 | 273.425 | 149.325 | 559.342 | 7.9 |

## 6. Windows Evaluation Results.

*6.1. Held-out selector behaviour.* T=65,536 wins calibration with a normalized score of 1.014. It is the largest candidate. Across the 18 held-out histories, its mean normalized latency is 1.012, against 1.101 for T=4,096. Agreement with the lower forced-path median is 18/18 and 12/18, respectively. The medians themselves contain measurement noise, so agreement is not classification accuracy against a known optimum. Every profile group is retained in Table 13 and Figure 8.

The result has a straightforward constraint: primary Windows intervals have at most 32,768 operations, so T=65,536 always takes log. Its calibration win cannot demonstrate a benefit from switching or locate a global crossover. Ratios can fall below one through timing variation even when hybrid and forced execution use equivalent paths. Read together with the adverse Linux control, this limits the claim we can make for selection. The experimental threshold does not change the implementation's default.

*Table 13*
*Held-out medians of three history medians; N/H/payload fixed, without a population interval*

| Locality | Operations | Forced log ms | Forced Merkle ms | H4096 ms | Calibrated ms |
|---|---|---|---|---|---|
| spread | 512 | 1.913 | 4.622 | 1.907 | 1.837 |
| spread | 4,096 | 25.714 | 38.878 | 26.550 | 27.312 |
| spread | 32,768 | 92.792 | 104.71 | 106.56 | 91.775 |
| repeated-key | 512 | 0.564 | 1.661 | 0.549 | 0.573 |
| repeated-key | 4,096 | 5.608 | 14.342 | 6.311 | 4.664 |
| repeated-key | 32,768 | 62.290 | 74.366 | 77.092 | 67.721 |

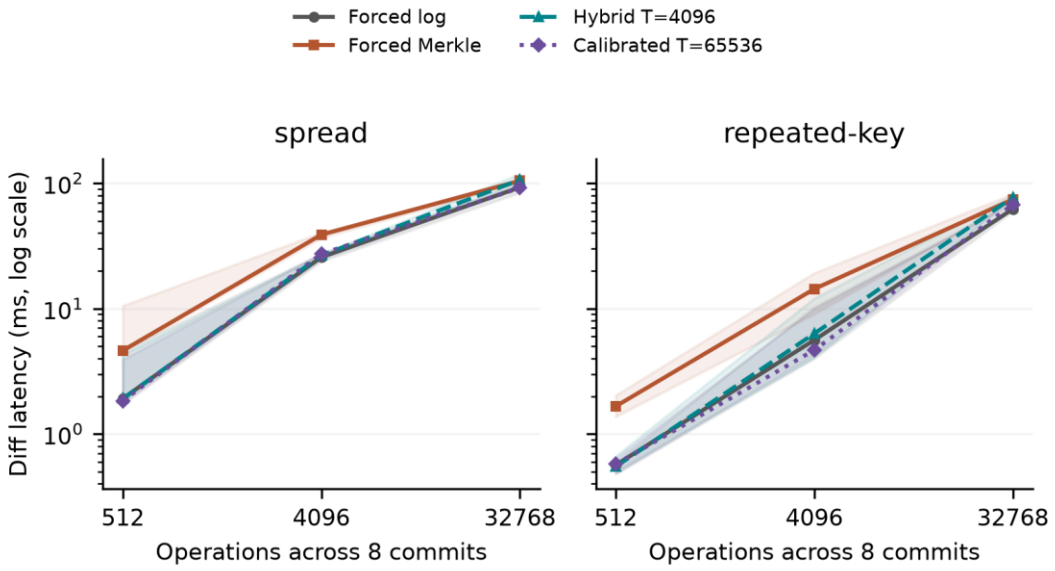



*Fig 8. Held-out selection at N=12,000 and H=8. Lines connect medians of three generated-history medians, with shading from their minimum to maximum. Each history supplies seven measured cached queries. The shading describes this range and is not a 95% confidence interval. The calibrated policy takes log at every operation count shown. The forced paths all use the same durable representation.*

*6.2. Sensitivity and resource interpretation.* Reading a detached public changeset view adds copying to the decision probe, which runs outside diff timing. We therefore cannot treat that probe as pure selector overhead. A separate diagnostic takes the prefix of the production function that validates versions, checks ancestry and counts operations internally, stopping before dispatch. For 18 held-out histories and two thresholds, it measures 252,000 warm calls. The median of case/threshold medians is 2.674 microseconds and the range is 1.611-3.678. Its decisions match production dispatch. Diff traversal and detached-view copying are excluded. This warm diagnostic provides neither a cold-cache estimate nor a causal correction to subtract from workflow times. It is separate from the 93-case workflow campaign.

The 21 sensitivity cases examine a prespecified factor or matched control over three seeds. The cases include payload=256 bytes, N=24,000, fixed-operation H=16, equal-capacity b4-d6/b16-d3 geometries and a C=8 hash-route/spread pair. Geometry changes reproduce the base history's state hashes while varying routing geometry alone. Fixing operations couples history length to changes per commit, which prevents estimating their separate effects in that comparison. The measurements characterize these profiles without selecting a generally best geometry.

For forced Merkle, median latency is 47.857 ms with more rows, 40.122 ms with longer histories and 39.950 ms with larger payloads. It is 43.004 ms for b4-d6 and 38.500 ms for b16-d3. Hash-route locality gives 0.174 ms, against 0.903 ms for its matched spread control. Each reported value is a median of three history medians. The summaries give per-history timings, selected paths and resource scopes. These are exploratory comparisons, with no claim of significance after multiple testing.

At T=4,096, the three sampler off/on pairs have median-latency ratios between 0.950 and 1.206. In every pair, the sampling-off run precedes the sampling-on run; cache state and background activity are uncontrolled. The ratios therefore do not identify a causal sampling penalty. We subtract no correction from other times. Resource counters cover the

whole worker. Tests retain observed zero I/O as a value, but Windows counters still identify neither database logical reads nor direct physical-disk traffic.

### *6.3. Public data and interface phases.*

*Table 14*
*Wine and CO2 workflow medians and full ranges of three repository medians; CO2 repeats one observed history*

| Source / system | Runs | Diff ms | Range ms | Import s | Persistent MiB |
|---|---|---|---|---|---|
| co2/revon-m | 3 | 0.612 | 0.562-0.661 | 0.033 | 1.191 |
| co2/revon-h | 3 | 0.309 | 0.302-0.318 | 0.035 | 1.184 |
| co2/dolt | 3 | 256.98 | 251.87-262.89 | 0.634 | 0.178 |
| wine/revon-m | 3 | 4.281 | 4.260-4.964 | 0.189 | 10.832 |
| wine/revon-h | 3 | 2.247 | 1.956-2.366 | 0.191 | 10.812 |
| wine/dolt | 3 | 296.76 | 246.75-302.81 | 0.819 | 1.360 |

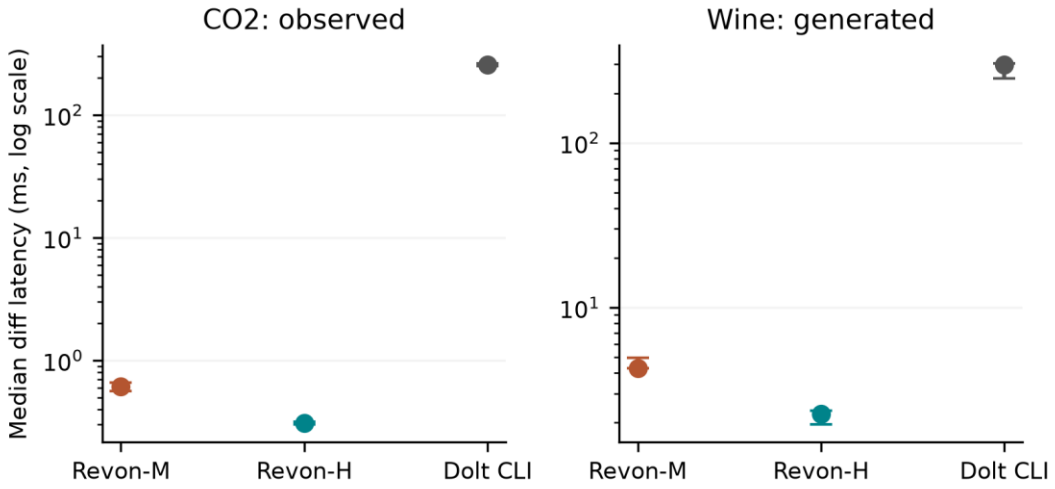



*Fig 9. Public-source workflows. A point is the median of three repository medians; a bar covers their full range. All CO2 repositories replay one observed publication history, whereas the wine repositories use three generated correction histories. There are seven cached queries per repository. The API/CLI boundaries differ, so the latency belongs to the workflow rather than an isolated storage engine.*

For co2-observed, the median Dolt command time is 256.063 ms. JSON interpretation takes 0.501 ms and common output construction 0.503 ms. The command accounts for 99.6% of median complete diff latency, but combines startup, native work and native JSON emission. This breakdown cannot attribute the whole command interval to the storage engine, nor justify subtracting a startup constant measured elsewhere.

For wine-generated, the corresponding medians are 288.464 ms for the Dolt command, 4.445 ms for JSON interpretation and 3.258 ms for common output construction. Command time is 97.2% of median complete diff latency and again includes startup, native work and native JSON emission. As in the CO2 case, engine cost is not isolated and there is no basis for subtracting an independent startup estimate.

### *6.4. Repeated Windows scale observations.*

*Table 15*
*Repeated Windows scale observations: medians of three seed/repository summaries, with full ranges retained in the evidence*

| Rows / system | Import s | Commit ms | Diff ms | Checkout s | Worker RSS GiB |
|---|---|---|---|---|---|
| 100,000/revon-m | 1.116 | 54.309 | 5.649 | 0.209 | 0.248 |
| 100,000/revon-h | 1.109 | 53.044 | 1.004 | 0.173 | 0.242 |
| 100,000/dolt | 1.662 | 759.99 | 283.65 | 0.662 | 0.223 |
| 1,000,000/revon-m | 10.614 | 88.233 | 33.694 | 1.862 | 1.939 |
| 1,000,000/revon-h | 10.738 | 85.362 | 1.013 | 1.575 | 1.925 |
| 1,000,000/dolt | 11.281 | 890.97 | 320.30 | 3.706 | 0.976 |

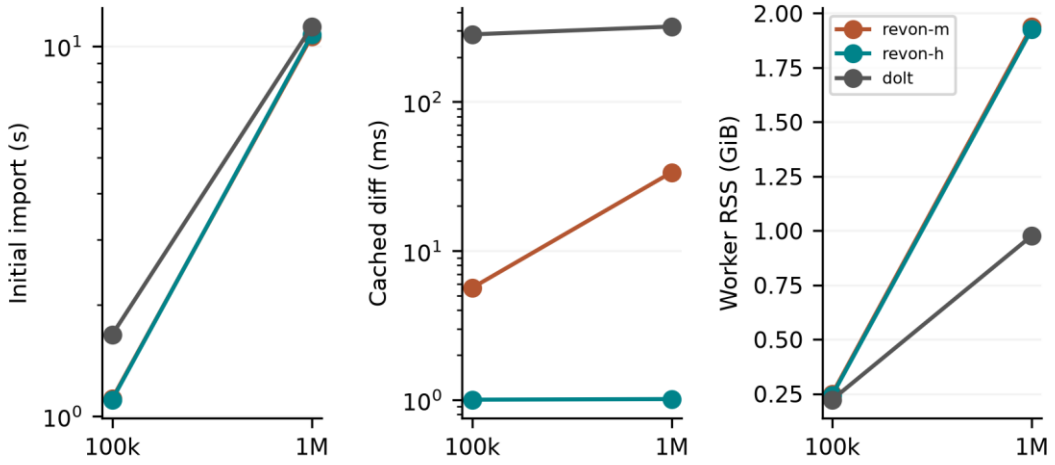



*Fig 10. Windows scale observations using three generated histories and fresh repositories for each size/system. The lines join descriptive medians at two sizes, rather than fitted asymptotic laws. Diff queries reuse cached state; import and checkout have separate timings. Peak RSS includes workload/oracle generation, storage work, verification and sampled descendants. The Linux million-row n=1 diagnostics belong to another campaign.*

The repeated Windows scale runs and Linux diagnostics are analysed separately. The three Windows histories start from a common synthetic distribution and apply different updates; they are not independent public datasets. Available memory and background processes are recorded in the manifest but remain uncontrolled. The numbers describe the tested workflows under those conditions. They cannot establish a population ranking or a general law of scaling.

*6.5. Answers to the research questions.* For RQ1, we observe trade-offs between latency, persistent storage and worker resources in complete workflows. Because the API/CLI boundaries differ, these observations cannot rank storage engines alone. For RQ2, log helps on several fixed histories, yet the Windows calibration winner uses it for every tested interval. A benefit from adaptive switching remains unestablished. For RQ3, controlled histories reveal effects of payload, routing and geometry within the staged design, without identifying a universal layout. Observed CO2 transitions, generated wine corrections and synthetic updates are different kinds of history and retain their respective scope limits.

**7. Discussion.** RQ1 is answered at the application interface. On Linux and Windows we measure import, commit, diff, checkout and persistent footprint, keeping campaign rows separate in analysis. Snapshot and Log-only reveal representation costs. Forced M, H and log comparisons keep Revon's representation fixed. Dolt offers a richer SQL/versioning interface, and its command-phase measurements make that workflow easier to inspect. They do not make native tree work equivalent to an isolated Python call.

For RQ2, the question is whether a selected parameter transfers to histories outside calibration. The Windows split fixes primary N, H, payload and geometry, and records selection before validation. The largest candidate behaves as always-log throughout the measured range. On Linux, the adverse threshold choice and dense-case parity show why fastest-path selection cannot be assumed. A more complex rule would require a separate calibration and untouched test histories. The findings give no reason to change the default threshold.

For RQ3, holding capacity equal still leaves room for differences in traversal depth and ancestor reconstruction, even when uniform routing gives equal expected occupancy. Payload encoding, touched leaves and output construction add latency too. The Windows cases use separate generated seeds and matched controls. The Linux geometry study holds one history fixed. Neither design estimates every interaction or an optimum across a population.

A valid diff comparison also depends on value semantics. Detaching inputs and outputs prevents mutable aliases from altering Revon's retained history. The v1 encoded identities keep true distinct from 1 and 1 distinct from 1.0, while leaving the object encoding and on-disk schema unchanged. Linux trials exercise those rules on the supplied histories. Unit tests additionally cover null versus absence, nested ownership, unsupported values and rollback.

**8. Limitations And Future Work.** External validity: the authors ran the Linux experiments on one cloud VM and the Windows experiments on one local host. This is neither independent-party replication nor a controlled OS comparison: hardware, runtime, virtualization and storage change together. Background services and filesystem caching stay active. The Windows manifest also records constrained available RAM and process conditions. The results do not describe a fully idle host or a cold cache.

Workload validity: TLC histories apply generated changes to one monthly release, and Linux scale diagnostics have n=1 per system. Windows validation uses three held-out generated seeds per primary profile, generated wine corrections and one observed CO2 publication sequence. Repeating that sequence in three CO2 repositories does not sample independent histories. We exclude a header-only revision before timing and keep unnamed columns without assigning them scientific meaning. Synthetic scale histories also share their initial distribution. Additional domains, observed histories and hardware are needed to generalize further.

Implementation scope: Revon has one writer, linear history and a fixed-geometry trie. Retained nodes, changesets and commits are loaded eagerly, with no eviction. Named branches, merge commits, conflict handling, schema evolution, remotes, ordered scans, crash fault injection and concurrent workloads are not evaluated. SQLite transaction atomicity by itself does not establish the complete set of durability and operational guarantees required for production.

Measurement and inference: the Dolt command interval includes startup, native work and JSON emission. Oracle and history state contribute to worker RSS. Sampling at ten-millisecond intervals may miss brief processes or peaks, and Windows I/O counters measure neither physical-disk traffic nor database logical reads. Sampler off/on order is

confounded with cache conditions as well. Linux bootstrap intervals describe repeated timings of fixed histories. Windows three-history ranges are descriptive and lack multiple-comparison correction. Source and semantic-hash checks establish internal consistency without independently reproducing timings.

Selection and design: primary Windows calibration tests four thresholds on two generated seeds and the specified profiles. T=65,536 is the largest candidate and does not switch in the tested range. Its result leaves the broader usefulness of adaptive selection unestablished. The staged sensitivity experiment is not fully factorial. At fixed operation count, H trades off against density, and hash-route cases require the matched low-density control. A different selector or crossover must be tested through a separately fixed experiment.

Storage could be extended with binary nodes, value chunks, packed objects, compression or reachability-based garbage collection. An ordered Merkle index or content-defined chunking could lead to a different design for range operations. Each such change needs its own semantic and performance evaluation. The fixed-trie measurements cannot predict the effects.

**9. Conclusion.** Revon makes it possible to compare two semantically consistent diff paths over a content-addressed fixed-trie history stored atomically in SQLite. Our contribution is an inspectable prototype and an empirical account of its workflow and selection costs. The Linux record contains 516 executions and a separate 72-execution supplement, retaining dense-case uncertainty and an adverse threshold choice. Windows contributes 93 worker cases and 3,402 checked queries across held-out histories, Wine Quality and CO2 sources, interface phases and repeated scale runs. Calibration chooses T=65,536, which takes log throughout the tested primary matrix. The optimal crossover therefore remains unresolved. These workloads show conditional trade-offs within the stated API/CLI, sampling, dataset and hardware limits. They do not establish a universal speed advantage or reproduction by an independent party.

Funding: No funding was received for this work.

Competing interests: The authors declare no competing interests.

Ethics and consent to participate: The study recruited no participants and involved no human or animal interventions. It uses synthetic histories and publicly released de-identified TLC records. No institutional ethics approval or exemption is claimed.

Consent for publication: Consent for publication does not apply because the paper contains no identifiable participant material.

Data and code availability: Our public inputs are NYC TLC taxi records [24], UCI Wine Quality [11] and the published CO2 package [12]. The code, executed source snapshots and experimental evidence are available at https://github.com/atharvasheersh/Revon. In paper/, the Linux reproducibility archive and the Windows reproducibility archive contain the accompanying materials. TLC input records and generated databases are not redistributed. Source metadata describes licences and transformations, and Appendix A gives access instructions.

**Acknowledgment.** We thank the open-source Dolt project and the dataset-versioning researchers whose work helped shape the experimental design.

# REFERENCES


[1] M. Akhlaghi et al. (2021). Toward long-term and archivable reproducibility, Computing in Science & Engineering, vol. 23, no. 3, pp. 82-91, https://doi.org/10.1109/MCSE.2021.3072860. [Online]. Available: https://arxiv.org/abs/2006.03018.

[2] M. Armbrust et al. (2020). Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores, Proc. VLDB Endowment, vol. 13, no. 12, pp. 3411-3424, https://doi.org/10.14778/3415478.3415560.

[3] Attic Labs. Noms Technical Overview. GitHub. [Online]. Available: https://github.com/attic-labs/noms/blob/master/doc/intro.md.

[4] A. Auvolat and F. Taïani (2019). Merkle Search Trees: Efficient State-Based CRDTs in Open Networks, in Proc. 38th IEEE Int. Symp. Reliable Distributed Syst. (SRDS), pp. 221-230, https://doi.org/10.1109/SRDS47363.2019.00032.

[5] J. Benet (2014). IPFS: Content Addressed, Versioned, P2P File System, arXiv:1407.3561, [Online]. Available: https://arxiv.org/abs/1407.3561.

[6] A. Bhardwaj et al. (2015). DataHub: Collaborative Data Science and Dataset Version Management at Scale, in Proc. 7th Biennial Conf. Innovative Data Syst. Research (CIDR), Open copy: https://www.cidrdb.org/cidr2015/Papers/CIDR15_Paper18.pdf.

[7] S. Bhattacherjee, A. Chavan, S. Huang, A. Deshpande, and A. Parameswaran (2015). Principles of Dataset Versioning: Exploring the Recreation/Storage Tradeoff, Proc. VLDB Endowment, vol. 8, no. 12, pp. 1346-1357, https://doi.org/10.14778/2824032.2824035.

[8] G. S. Brodal, S. Sioutas, K. Tsakalidis, and K. Tsichlas (2020). Fully persistent B-trees, Theoretical Computer Science, vol. 841, pp. 10-26, https://doi.org/10.1016/j.tcs.2020.06.027. [Online]. Available: https://cs.au.dk/~gerth/papers/tcs20.pdf.

[9] P. Buneman, S. Khanna, and W.-C. Tan (2001). Why and where: A characterization of data provenance, in Database Theory: ICDT Lecture Notes in Computer Science, vol. 1973, 2001, pp. 316-330, https://doi.org/10.1007/3-540-44503-X_20. [Online]. Available: https://www.pure.ed.ac.uk/ws/files/16509989/Why_and_Where_A_Characterization_of_Data_Provenance.pdf.

[10] S. Chacon and B. Straub (2014). Git Internals: Git Objects, in Pro Git, 2nd ed., [Online]. Available: https://git-scm.com/book/en/v2/Git-Internals-Git-Objects.

[11] P. Cortez, A. Cerdeira, F. Almeida, T. Matos and J. Reis (2009). Wine Quality [Dataset], UCI Machine Learning Repository, https://doi.org/10.24432/C56S3T. CC BY 4.0.

[12] DataHub Contributors (2026). CO2 PPM - Trends in Atmospheric Carbon Dioxide, NOAA-derived published data package, accessed October https://github.com/datasets/co2-ppm. ODC-PDDL-1.0; acquired revision identifiers and file hashes are retained in the experiment manifest.

[13] DoltHub (2026). Dolt Storage Engine: Block Store, Dolt Documentation, accessed Oct. 2, [Online]. Available: https://www.dolthub.com/docs/architecture/storage-engine/block-store/.

[14] J. R. Driscoll, N. Sarnak, D. D. Sleator, and R. E. Tarjan (1989). Making data structures persistent, J. Comput. Syst. Sci., vol. 38, no. 1, pp. 86-124, https://doi.org/10.1016/0022-0000(89)90034-2.

[15] J. Edwards and T. Petricek (2025). Baseline: Operation-based evolution and versioning of data, arXiv:2512.09762, preprint, updated 2026. [Online]. Available: https://arxiv.org/abs/2512.09762.

[16] A. González-Cebrián, M. Bradford, A. E. Chis, and H. González-Vélez (2024). Standardised versioning of datasets: A FAIR-compliant proposal, Scientific Data, vol. 11, art. 358, https://doi.org/10.1038/s41597-024-03153-y. [Online]. Available: https://www.nature.com/articles/s41597-024-03153-y.

[17] H. Gou, Z. Zhang, Y. Sun, P. Xu, F. Tian, L. Wang, and J. Wang (2026). Git4Data: Database-native version control for AI agents, arXiv:2609.02106, preprint. [Online]. Available: https://arxiv.org/abs/2609.02106.

[18] A. Guo, J. Li, P. Sukprasert, S. Khuller, A. Deshpande, and K. Mukherjee (2024). To store or not to store: A graph theoretical approach for dataset versioning, in Proc. IEEE Int. Parallel and Distributed Processing Symposium (IPDPS), pp. 479-493, https://doi.org/10.1109/IPDPS57955.2024.00049. [Online]. Available: https://arxiv.org/abs/2402.11741.

[19] S. Huang, L. Xu, J. Liu, A. J. Elmore, and A. Parameswaran (2017). OrpheusDB: Bolt-on Versioning for Relational Databases, Proc. VLDB Endowment, vol. 10, no. 10, pp. 1130-1141, https://doi.org/10.14778/3115404.3115417.

[20] M. Maddox et al. (2016). Decibel: The Relational Dataset Branching System, Proc. VLDB Endowment, vol. 9, no. 9, pp. 624-635, https://doi.org/10.14778/2947618.2947619.

[21] F. McSherry, D. G. Murray, R. Isaacs, and M. Isard (2013). Differential dataflow, in Proc. 6th Biennial Conf. Innovative Data Systems Research (CIDR), [Online]. Available: https://www.cidrdb.org/cidr2013/Papers/CIDR13_Paper111.pdf.

[22] R. C. Merkle (1988). A digital signature based on a conventional encryption function, in Advances in Cryptology: CRYPTO '87, Lecture Notes in Computer Science, vol. 293, pp. 369-378, https://doi.org/10.1007/3-540-48184-2_32.

[23] A. Muthitacharoen, B. Chen, and D. Mazières (2001). A low-bandwidth network file system, in Proc. 18th ACM Symp. Operating Syst. Principles (SOSP), pp. 174-187, https://doi.org/10.1145/502034.502052.

[24] New York City Taxi and Limousine Commission (2024). TLC Trip Record Data: Yellow Taxi, January public dataset, accessed Oct. 1, 2026. [Online]. Available: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page.

[25] N. Saquib, C. Krintz, and R. Wolski (2023). Replicated versioned data structures for wide-area distributed systems, IEEE Transactions on Parallel and Distributed Systems, vol. 34, no. 1, pp. 207-224, https://doi.org/10.1109/TPDS.2022.3217969. [Online]. Available: https://racelab.cs.ucsb.edu/papers/tpds22-versioned.pdf.

[26] M. E. Schüle, L. Karnowski, J. Schmeißer, B. Kleiner, A. Kemper, and T. Neumann (2019). Versioning in main-memory database systems: From MusaeusDB to TardisDB, in Proc. 31st Int. Conf. Scientific and Statistical Database Management (SSDBM), pp. 169-180, https://doi.org/10.1145/3335783.3335792. [Online]. Available: https://www-db.cs.tum.edu/~schuele/data/tardis.pdf.

[27] A. Seering, P. Cudré-Mauroux, S. Madden, and M. Stonebraker (2012). Efficient versioning for scientific array databases, in Proc. IEEE 28th Int. Conf. Data Engineering (ICDE), pp. 1013-1024, https://doi.org/10.1109/ICDE.2012.102. [Online]. Available: https://dspace.mit.edu/entities/publication/a5c2c050-6abd-47c1-8bee-4f8c51b6a15f.

[28] T. Sehn (2024). Dolt's Storage Engine, DoltHub Blog, Feb. 29, [Online]. Available: https://www.dolthub.com/blog/2024-02-29-storage-engine/.

[29] R. Shraga and R. J. Miller (2023). Explaining Dataset Changes for Semantic Data Versioning with Explain-Da-V, Proc. VLDB Endow., vol. 16, no. 6, pp. 1587-1600, https://doi.org/10.14778/3583140.3583169.

[30] E. Soroush and M. Balazinska (2013). Time travel in a scientific array database, in Proc. IEEE 29th Int. Conf. Data Engineering (ICDE), pp. 98-109, https://doi.org/10.1109/ICDE.2013.6544817. [Online]. Available: https://scidb.cs.washington.edu/paper/ICDE13_conf_full_422.pdf.

[31] A. Twigg, A. Byde, G. Miłos, T. Moreton, J. Wilkes, and T. Wilkie (2011). Stratified B-trees and versioning dictionaries, in Proc. 3rd USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage), [Online]. Available: https://arxiv.org/abs/1103.4282.

[32] S. Wang et al. (2018). ForkBase: An Efficient Storage Engine for Blockchain and Forkable Applications, Proc. VLDB Endowment, vol. 11, no. 10, pp. 1137-1150, https://doi.org/10.14778/3231751.3231762.

[33] J. Zhao and S. Krishnan (2024). Compression and in-situ query processing for fine-grained array lineage, arXiv:2405.17701, [Online]. Available: https://arxiv.org/abs/2405.17701.

[34] J. Klump, H. Pampel, L. Rothfritz, and D. Strecker (2024). Recommendations on Data Versioning. Zenodo. https://doi.org/10.5281/zenodo.13743875

[35] D. Trautwein, A. Raman, G. Tyson, I. Castro, W. Scott, M. Schubotz, et al. (2022). Design and evaluation of IPFS. Proceedings of the ACM SIGCOMM 2022 Conference, pp. 739-752. https://doi.org/10.1145/3544216.3544232

[36] A. Arasu, B. Chandramouli, J. Gehrke, E. Ghosh, D. Kossmann, J. Protzenko, et al. (2021). FastVer: Making Data Integrity a Commodity. Proceedings of the 2021 International Conference on Management of Data, pp. 89-101. https://doi.org/10.1145/3448016.3457312

[37] E.D. Demaine, S. Langerman, E. Price (2010). Confluently Persistent Tries for Efficient Version Control. Algorithmica, vol. 57, no. 3, pp. 462-483. https://doi.org/10.1007/S00453-008-9274-Z

[38] S. Lanka, E. Mays (1991). Fully persistent B+-trees. ACM SIGMOD Record, vol. 20, no. 2, pp. 426-435. https://doi.org/10.1145/119995.115861

[39] G.S. Brodal, C.M. Rysgaard, R. Svenning (2023). External Memory Fully Persistent Search Trees. Proceedings of the 55th Annual ACM Symposium on Theory of Computing, pp. 1410-1423. https://doi.org/10.1145/3564246.3585140

[40] C. Allain, B. Clément, A. Moine, G. Scherer (2024). Snapshottable Stores. Proceedings of the ACM on Programming Languages, vol. 8, no. ICFP, pp. 338-369. https://doi.org/10.1145/3674637

[41] A. Miller, M. Hicks, J. Katz, E. Shi (2014). Authenticated data structures, generically. Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, pp. 411-423. https://doi.org/10.1145/2535838.2535851

[42] V. Salis, D. Spinellis (2019). RepoFS: File system view of Git repositories. SoftwareX, vol. 9, pp. 288-292. https://doi.org/10.1016/j.softx.2019.03.007

[43] A. Bhardwaj, A. Deshpande, A.J. Elmore, D. Karger, S. Madden, A. Parameswaran, et al. (2015). Collaborative data analytics with DataHub. Proceedings of the VLDB Endowment, vol. 8, no. 12, pp. 1916-1919. https://doi.org/10.14778/2824032.2824100

[44] A. Chavan, A. Deshpande (2017). DEX. Proceedings of the 2017 ACM International Conference on Management of Data, pp. 171-186. https://doi.org/10.1145/3035918.3064056

[45] J.F. Roddick (1995). A survey of schema versioning issues for database systems. Information and Software Technology, vol. 37, no. 7, pp. 383-393. https://doi.org/10.1016/0950-5849(95)91494-K

[46] G. Ozsoyoglu, R. Snodgrass (1995). Temporal and real-time databases: a survey. IEEE Transactions on Knowledge and Data Engineering, vol. 7, no. 4, pp. 513-532. https://doi.org/10.1109/69.404027

[47] C. De Castro, F. Grandi, M.R. Scalas (1997). SCHEMA VERSIONING FOR MULTITEMPORAL RELATIONAL DATABASES. Information Systems, vol. 22, no. 5, pp. 249-290. https://doi.org/10.1016/S0306-4379(97)00017-3

[48] T.J. Green, G. Karvounarakis, V. Tannen (2007). Provenance semirings. Proceedings of the twenty-sixth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems, pp. 31-40. https://doi.org/10.1145/1265530.1265535

[49] Y.L. Simmhan, B. Plale, D. Gannon (2005). A survey of data provenance in e-science. ACM SIGMOD Record, vol. 34, no. 3, pp. 31-36. https://doi.org/10.1145/1084805.1084812

[50] L. Carata, S. Akoush, N. Balakrishnan, T. Bytheway, R. Sohan, M. Seltzer, et al. (2014). A primer on provenance. Communications of the ACM, vol. 57, no. 5, pp. 52-60. https://doi.org/10.1145/2596628

[51] B. Glavic (2021). Data Provenance. Foundations and Trends in Databases, vol. 9, no. 3-4, pp. 209-441. https://doi.org/10.1561/1900000068

[52] Y. Amsterdamer, D. Deutch, V. Tannen (2011). Provenance for aggregate queries. Proceedings of the thirtieth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems, pp. 153-164. https://doi.org/10.1145/1989284.1989302

[53] F. Jamil, A. Khan, A. Anjum, M. Ahmed, F. Jabeen, N. Javaid (2018). Secure provenance using an authenticated data structure approach. Computers & Security, vol. 73, pp. 34-56. https://doi.org/10.1016/j.cose.2017.10.005

[54] M.D. Wilkinson, M. Dumontier, I.J. Aalbersberg, G. Appleton, M. Axton, A. Baak, et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. Scientific Data, vol. 3, no. 1, article 160018. https://doi.org/10.1038/sdata.2016.18

[55] B. Pan, N. Stakhanova, S. Ray (2023). Data Provenance in Security and Privacy. ACM Computing Surveys, vol. 55, no. 14s, pp. 1-35. https://doi.org/10.1145/3593294

[56] L. Moreau, P. Groth, J. Cheney, T. Lebo, S. Miles (2015). The rationale of PROV. Journal of Web Semantics, vol. 35, pp. 235-257. https://doi.org/10.1016/j.websem.2015.04.001

[57] K. Wolstencroft, S. Owen, O. Krebs, Q. Nguyen, N.J. Stanford, M. Golebiewski, et al. (2015). SEEK: a systems biology data and model management platform. BMC Systems Biology, vol. 9, no. 1, article 33. https://doi.org/10.1186/s12918-015-0174-y

[58] D. Comer (1979). Ubiquitous B-Tree. ACM Computing Surveys, vol. 11, no. 2, pp. 121-137. https://doi.org/10.1145/356770.356776

[59] Y.S. Kwong, D. Wood (1980). On B-trees. Proceedings of the 1980 ACM SIGMOD international conference on Management of data - SIGMOD '80, pp. 207. https://doi.org/10.1145/582250.582282

[60] W.J. Hansen (1981). A Cost Model for the Internal Organization of B+-Tree Nodes. ACM Transactions on Programming Languages and Systems, vol. 3, no. 4, pp. 508-532. https://doi.org/10.1145/357146.357152

[61] M.A. Bender, E.D. Demaine, M. Farach-Colton (2005). Cache-Oblivious B-Trees. SIAM Journal on Computing, vol. 35, no. 2, pp. 341-358. https://doi.org/10.1137/S0097539701389956

[62] P. O'Neil, E. Cheng, D. Gawlick, E. O'Neil (1996). The log-structured merge-tree (LSM-tree). Acta Informatica, vol. 33, no. 4, pp. 351-385. https://doi.org/10.1007/s002360050048

[63] F. Chang, J. Dean, S. Ghemawat, W.C. Hsieh, D.A. Wallach, M. Burrows, et al. (2008). Bigtable. ACM Transactions on Computer Systems, vol. 26, no. 2, pp. 1-26. https://doi.org/10.1145/1365815.1365816

[64] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, et al. (2007). Dynamo. Proceedings of twenty-first ACM SIGOPS symposium on Operating systems principles, pp. 205-220. https://doi.org/10.1145/1294261.1294281

[65] M. Rosenblum, J.K. Ousterhout (1992). The design and implementation of a log-structured file system. ACM Transactions on Computer Systems, vol. 10, no. 1, pp. 26-52. https://doi.org/10.1145/146941.146943

[66] L. Lu, T.S. Pillai, H. Gopalakrishnan, A.C. Arpaci-Dusseau, R.H. Arpaci-Dusseau (2017). WiscKey. ACM Transactions on Storage, vol. 13, no. 1, pp. 1-28. https://doi.org/10.1145/3033273

[67] H.T. Vo, S. Wang, D. Agrawal, G. Chen, B.C. Ooi (2012). LogBase. Proceedings of the VLDB Endowment, vol. 5, no. 10, pp. 1004-1015. https://doi.org/10.14778/2336664.2336673

[68] C. Riegger, I. Petrov (2024). Storage Management with Multi-Version Partitioned BTrees. Information Systems, vol. 125, pp. 102403. https://doi.org/10.1016/j.is.2024.102403

## APPENDIX A: REPRODUCIBILITY

The Revon GitHub repository contains the code and experiment records for this paper. Its paper/ directory contains the Linux reproducibility archive and the Windows reproducibility archive. The README files cover dependency setup, acquisition of public data, execution of experiments and verification of the evidence [1]. Each bundle retains its executed sources, fixed protocol, raw measurements and internal audits.

Analyse the primary Linux, supplementary Linux and Windows campaigns separately because their protocols differ. The preparation instructions allow TLC inputs and generated repositories to be rebuilt. Public-source metadata supplies licences, acquisition versions, checksums and the CO2 exclusion made before timing. To preserve the collected records, run auditors against disposable copies and localize the recorded data paths. Matching semantic hashes verifies consistency, not elapsed times or identical timing on another machine.

## APPENDIX B: METHODS AND EVIDENCE DETAILS

The Linux bundles record the executed sources and hashes alongside environments, fixed workloads, execution order, raw observations and audits. Workers use PYTHONHASHSEED=20261008. Before measurement, the primary and supplementary campaigns pass 67 and 69 unit tests, respectively. During supplement preflight, an incomplete transfer omits the test_data directory. Measurement begins only after that empty fixture directory is restored and the complete suite passes. The recorded execution history retains the setup failure.

A primary execution begins with a fresh worker and repository construction. Within repetition blocks, model order is deterministically shuffled and rotated. Import ends at the creation of the first durable version. Incremental-commit latency is the median of that history's individual commit timers. Diff compares the first and last versions and emits sorted changed keys with old/new-value hashes in canonical JSON. Checkout emits the complete sorted key/value state in canonical UTF-8 JSON. Dolt timing covers CLI launch, JSON parsing and output normalization; Revon uses an in-process API. The boundaries differ, so the resulting workflow ratios cannot isolate engine speed.

Correctness verification is outside operation timing. Every primary execution checks initial and final state hashes, all H+1 historical states, and the observed canonical diff against a separately constructed workload oracle. Both expected and observed hashes are retained. The audit regenerates workloads, checks matrix membership, selected paths and source/data/protocol identities, and recomputes summaries. We also audit downloaded

records locally. These checks establish internal integrity rather than an independent repeat of the timing experiment.

After warm-ups are excluded, the seven measured repository trials are summarized with medians and interquartile ranges (IQRs). We pair matched trials before computing a median latency ratio and a paired percentile-bootstrap interval from 20,000 resamples. A ratio above one favours the denominator. These exploratory 95% intervals concern repeated timings of one fixed workload on one VM, with no population or multiple-comparison interpretation. The median of paired ratios need not equal the ratio of the latency medians. Every completed trial is retained, including timing outliers.

Every 10 ms, a parent process samples worker and descendant RSS, CPU and I/O. Workload generation, oracle state, verification and storage work all contribute to peaks, preventing attribution to the storage engine alone. A brief peak or short-lived subprocess may fall between samples. The primary Linux sampler records an all-zero read counter as missing, so its records permit no conclusion about read I/O. The supplementary campaign separates measured zero from unavailable counters and is analysed on its own. Logical examination of cached data can occur with zero recorded physical read bytes.

Linux footprint is directory size before closing or compaction, including indexes, metadata and representation costs. The records additionally provide logical-payload normalization and selected first-trial compaction probes. These are complete-workflow measurements, not isolated tree overhead. CPU, RSS and I/O span the worker lifecycle, beyond the boundaries used for individual operation timers.

The auxiliary Windows threshold experiment specifies non-overlapping calibration and validation profiles in advance. Its operation-count candidates are 1,024, 2,048, 4,096, 8,192 and 16,384. Minimizing normalized latency during calibration selects T=16,384 before validation. That threshold gives a held-out normalized score of 1.032, compared with 1.297 for T=4,096. Always-log scores 1.000 on both splits. The cached-query results belong to the source and environment recorded for that experiment and remain separate from Linux timing. Winning as the largest candidate does not establish an optimum. Long-history and three-seed pilot records are exploratory context.

In the Linux threshold control we hold T=16,384 fixed over 10,000 rows and 512 changes per commit. H=16 and H=64 give 8,192 and 32,768 accumulated operations. Forced log, forced Merkle and hybrid each use separate workers, two warm-ups and seven measured trials. Linux outcomes do not select the threshold. These cases test transfer of the auxiliary calibration choice, rather than carrying out a new calibration/validation cycle.

The Linux geometry workload uses 10,000 rows, ten commits, 100 changes per commit, 32-byte synthetic values and seed 20260824. We force Merkle diff for b4-d6, b8-d3, b8-d4, b8-d5 and b16-d3. Each configuration has two warm-ups and seven measured trials in fresh workers. Order rotates to limit confounding between time and configuration. Every execution verifies eleven historical states and the endpoint diff outside timing. The adapter keeps the workflow experiment's output contract. Peaks observed by the geometry adapter's in-process monitor are reported separately from external RSS measurements.

The Windows protocol and case manifest assign 202610161/202610162 to calibration and 202610171-202610173 to validation/sensitivity. Workers set PYTHONHASHSEED=20261016. Monitoring uses 202610190, generated public corrections use 202610201-202610203, and scale uses 202610221-202610223. A worker's queries share its repository, while every worker builds fresh persistent state. The threshold is timestamped before validation and selection is not repeated afterwards. We do not pool Linux/Windows rows.

The Windows instrumentation uses the native Dolt JSON-diff command and the adapter's parser contract, timing the phases explicitly. Before measurement, preflight

compares actual output with the native adapter and the independent oracle. Revon checks observed output hashes, all historical-state hashes, reverse/identity requests, object integrity and reopening outside timing. Setup checks and data-quality exclusions are documented and applied before measurement. The retained query matrix and timing-source hashes identify the conditions under which execution took place.

Run the portable audit on a disposable evidence copy. It opens a fresh interpreter, imports the executed source and localizes the recorded public-data path. It regenerates workloads and state/diff oracles, checks every configuration, trial and order identity, and recomputes numerical summaries. It then verifies the hashes of collected files. The archives provide the checks and data-acquisition recipes needed to verify consistency. To reproduce timings, the experiments themselves must be run.

*Table 16*
*Public-data checksums and the Linux protocol*

| Item | Definition |
|---|---|
| Source | NYC TLC yellow_tripdata_2024-01.parquet; 2,964,624 source rows |
| Raw SHA-256 | c4d59da7bbc8abaeeeb1727947ee93d9891a71acb42854bd80db1571b2030510 |
| Prepared million-row SHA-256 | 6608b7680e82432ca22f90a71d4994676ffd290c4fc85f17f91fab03d4c7b661 |
| P1 | 100,000 records; H=10; C=100; 2 warm-ups + 7 measured trials per system |
| Scale diagnostic | 1,000,000 records; H=10; C=1,000; one trial per system; no interval or ranking |

*Table 17*
*Windows execution matrix; cached queries are separate from independent repetitions of histories or repositories*

| Component | Fresh workers | Histories / control | Warm-up queries | Measured queries |
|---|---|---|---|---|
| Calibration | 12 | 2 seeds × 2 localities × 3 densities | 144 | 504 |
| Held-out | 18 | 3 different seeds × same profiles | 216 | 756 |
| Sensitivity | 21 | 3 seeds × 7 matched variations | 252 | 882 |
| Monitor diagnostic | 6 | 3 off/on pairs, one fixed history | 72 | 252 |
| Public sources | 18 | 2 sources × 3 runs × M/H/Dolt | 36 | 126 |
| Scale | 18 | 2 sizes × 3 seeds × M/H/Dolt | 36 | 126 |
| Total | 93 | Different campaigns are never pooled | 756 | 2646 |